CANVAS GAME ARCHIVE

五款单文件游戏
开发过程与借鉴分析

DEVELOPMENT RETROSPECTIVE & DESIGN REFERENCES

从 13,928 行内联 JavaScript 出发，复盘一套
「判据驱动 + AI 接管 + 版本留档」的个人游戏工程方法

版本 **v1.3.0** · 快照 **2026-10-19** · 提交 **a7d9c3e**

五款游戏 · 183 项自动判据 · 21 秒全仓回归

线上 **kevin0521-wjw.github.io/canvas-games**

归档于 2026-10-20
Canvas 游戏档案馆

目 录

01 项目概览 五款游戏、规模、线上入口 P3

02 开发过程：五个版本 从 92 项判据到 183 项 P5

03 架构：为什么五款长一个样 单帧流水线、累加器、输入合并 P7

04 逐款实现拆解 五款各自真正有技术含量的地方 P10

05 借鉴来源 原神 / 东方 / 元气骑士 / Breakout / 雷电 P16

06 验证方法论 判据、AI 反作弊、截图复核 P18

07 缺陷档案 修复清单与尚未修复的三处 P20

08 交付与运行 目录结构、起服务、跑判据 P22

09 附录：文件地图 一表看完整仓库 P24

01 项目概览

> 这不是五款「能跑就行」的练习作。它们共用一套刻意统一的骨架：固定逻辑分辨率、单帧状态对象、输入合并层、固定步长累加器、分层渲染，每一款都配一份能自动跑出结论的判据脚本。 本文要回答三件事：**它是怎么做出来的、它借了谁的东西、凭什么说它是对的。**

1.1 五款游戏

游戏	类型	行数	体积	逻辑分辨率	代码风格	判据	一句话
星际防线 · 初版 `space-line-v1`	纵版射击	5,619	240 KB	800×600	ES class ×13 + CFG 配置表	14	唯一一款「配置驱动」写法，三把武器是同一张表的三行
符箓灵弹 · 巧阵 `talisman-pinball`	弹球 × 元素反应	1,619	64 KB	520×880	纯函数 + IIFE	29	一面符阵墙缓缓下压，火水雷风四元素反应
星屑符咒 · 弹幕手记 `touhou-danmaku`	弹幕射击	2,334	107 KB	480×640	纯函数	40	算法密度最高：闭式最近接近点求威胁，不靠采样
深渊回响 `soul-knight`	俯视角 roguelite	1,990	85 KB	640×448	纯函数	44	房间是线性数组，不是迷宫图；一层层往下潜
星轨防线 v2 `starlane-v2`	纵版射击	2,366	113 KB	720×480	纯函数	56	缝合三套系统：关卡链、升级、护盾朝向
合计		**13,928**	**609 KB**	全部零外部依赖、全部单文件		**183**	全仓回归 21 秒

13,928	183	21s	0	5/5
行内联 JS / CSS / HTML	项自动判据	全仓回归耗时	外部依赖 / 构建步骤	判据全绿

1.2 什么算「单文件」

每款游戏的 `index.html` 同时包含样式、画布、全部游戏逻辑，双击即可运行，没有任何 `<script src>`、没有 npm、没有构建产物、没有 CDN。这带来两个真实的好处： 一是**没有版本漂移**（判据读到的一定是玩家跑的那份代码），二是**归档成本为零**（一个文件就是一款游戏）。 代价同样明显：没有模块系统、没有热更新、状态全在一处，改一个数值要看住三处引用。

1.3 线上入口

页面	地址	状态
档案馆首页（版本记录）	kevin0521-wjw.github.io/canvas-games/	200
星际防线 · 初版	…/canvas-games/space-line-v1/	200
符箓灵弹 · 巧阵	…/canvas-games/talisman-pinball/	200
星屑符咒 · 弹幕手记	…/canvas-games/touhou-danmaku/	200
深渊回响	…/canvas-games/soul-knight/	200
星轨防线 v2	…/canvas-games/starlane-v2/	200

仓库 `github.com/kevin0521-wjw/canvas-games` 本身不能玩（GitHub 只按文本渲染 .html）， 能玩的是上面的 GitHub Pages。Pages 是仓库的下游只读镜像：别人可以看、可以玩，但不能改， 要协作只能 Fork + PR。

02 开发过程：五个版本

> 整个项目经历了五个留档版本。有意思的是，**每一版最有价值的产出都不是新功能，而是一个被证伪的假设** —— 「判据全绿」这件事本身，在四个版本里被推翻过四次。

2.1 时间线

版本	日期	判据数	这一版真正发生了什么
v1.0.0	10-12	92	四款游戏跑到判据全绿（15 / 22 / 24 / 31），第一次给 `games/` 建快照。 **此前这个目录根本不在版本控制里**，所以更早的状态没有东西可以回填 —— 这件事本身就是教训。
v1.1.0	10-13	约 105	加 AI 接管与二倍速、重算平衡、修「开始按钮点不到」的布局 bug。同时补录了会话最初那一版。 但真正有价值的是：**平衡性探针炸出三个真 bug**，全都藏在「31 项断言全绿」后面 —— 比如星轨 v2 在第 3 关会直接卡死。
v1.2.0	10-13	120	修一个影响**全部五份判据脚本**的收尾 bug（每次 smoke 白等 3~5 分钟），再用第三代平衡探针 逐条验证上一版的下调成果 —— 结果三条数据全部不可信，查清后有一个真 bug、一次改动过头、 以及三条判据自己有缺陷。
v1.2.1	10-15	120	用户一句「深色模式改掉，不好看」，把档案页从深蓝底改成浅色。真正花时间的不是换背景色， 而是**深色主题那套亮色强调色在白底上等于看不见**，整组要重挑；顺手揪出两个只在成品里才看得见的 bug。
v1.3.0	10-19	183	按八条要求全面整改：去抖屏闪屏、符箓灵弹整款重做、五款补齐 AI 与二倍速、星屑符咒弹幕重做、 深渊回响改无限下潜。判据 120 → 183。

2.2 v1.3.0 的八条整改

这一版是本文的重点，因为它把「玩家感受」直接翻译成了可执行的技术指标：

#	原话	落地方案	量化验收
1	UI 和颜色修改，调亮	符箓灵弹换宣纸暖白底；其余四款面板 / HUD / 次级文字提亮一档	正文对比度 ≥ 4.5:1（WCAG）
2	大量搜索原神相关资源	取公开资料里的元素反应机制与量级，建成游戏内反应对照表；美术全部程序化重绘	反应表逐条进判据，伤害值对照
3	每个游戏都要二倍速和 AI 接管	五款补齐；AI 只写方向键布尔值，与键盘输入合并后走同一套物理	单帧位移 ≤ 玩家移速；判定半径开关前后逐项相同
4	星屑符咒 AI 都通不了关	弹幕重做：每张符卡留预告期、弹速分层、禁止多组矛盾封路同时出现	最差安全格 9.4% → **53.8%**；AI 9/9 通关
5	一关结束之后应该还有下一层	深渊回响 `run.depth++` 无限下潜；其余补关卡链	连续下潜 3 层不崩、不结束
6	取消闪屏和屏幕抖动	五款全删 `run.shake` / `run.flash`，改为屏幕边缘径向脉冲	源码特征串扫描命中即红
7	特效 UI 贴原神美工	装裱金框 + 四角云勾、玉牌质感符块、纸符造型灵弹、朱砂阵线	截图逐帧复核（抓到 5 处）
8	借鉴元素反应；打「缓慢下压的墙」而非满屏弹幕	整款重做：8 列 × 2~5 排符阵墙下压，火水雷风四元素	全屏敌弹 **0 发**；BOSS 符矢峰值 3（硬上限 4）

一个判断：把「不好玩」翻译成指标

「AI 都通不了关」这句话本身不是需求。能落地的问法是：**每一帧是否至少存在一条走得通的通道？** 把整屏离散成格子，逐帧统计「从当前位置出发、仍能走到的安全格占比」，最差的那一帧就是难度上限。 实测最差帧只有 9.4% —— 也就是说某些帧里，玩家确实无路可走，这不是「难」，是设计错误。 重做后这个数字变成 53.8%，AI 才有 9/9 通关。

2.3 留档这件事本身

`versions/2026-10-12`、`2026-10-13`、`2026-10-19` 三个目录各存了 五份源码 + 判据脚本 + 当次回归日志。它们的作用不是「纪念」：v1.3.0 里那次整款重做， 之所以敢把 1,142 行的符箓灵弹推翻，是因为旧版本、旧判据、旧日志都在，随时能对照「原来那版到底是什么手感」。

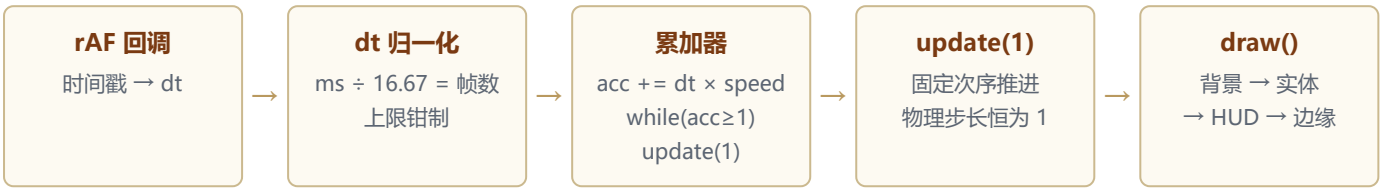
踩过的坑：快照根目录是 404

`versions/2026-10-19/` 这类目录里没有 `index.html`，而 GitHub Pages 不生成目录列表页，所以根地址直接 404。要验某一版必须写全路径，例如 `.../versions/2026-10-19/talisman-pinball/`。这类「离线对着文件系统觉得理所当然、 上线就断」的问题，只有真的用 HTTP 探一遍才会暴露。

03 架构：为什么五款长一个样

> 五款游戏的品类差得很远（纵版射击、弹球、弹幕、俯视角 roguelite），但骨架几乎一模一样。 这是刻意的：**骨架统一，问题就只会出在各自的「更新层」里**，排查范围立刻缩小一半。

3.1 单帧流水线



三条规则贯穿全部五款：

- **固定逻辑分辨率。**CSS 只改显示尺寸，画布坐标永不换算。好处是所有数值（碰撞半径、间距、速度） 都是「设计值」，不会因为窗口大小变得不可复现。
- `update` **永远只收到 1。**二倍速是「多跑几次」，不是「一次跑两步」。这条后面单独讲。
- **渲染不写状态。**`draw()` 只读 `run`，这保证了判据可以在完全不渲染的情况下 把一局推演到底（几万帧只要几百毫秒）。

3.2 一局的状态对象

每款的 `newRun()` 返回一个纯数据对象，一局的全部状态都挂在这一个根上： 实体数组、计时器、分数、随机种子、开关状态。这么做的直接收益是**「重开」等于换一个对象**， 没有残留状态可言。代价是判据必须拿对引用 —— 这一点后来真的坑了 16 条断言。

```
function newRun() {
  return {
    t: 0, over: false, paused: false, acc: 0, speed: _speed,
    hp: 3, score: 0, wave: 0,
    el: 'pyro',            // 当前元素
    blocks: [],            // 符阵 = 一组带元素的符块
    ball: null, projs: [], arcs: [], parts: [], floats: [],
    edge: { v: 0, rgb: '192,68,28' },   // 边缘脉冲：只有强度与颜色
    reactions: {},         // 本局各反应的触发次数
    rest: 1.1,             // 开阵前的缓冲（秒）
    wallY: 0, wallFade: 0, wallSpeed: 12,
  };
}
```

3.3 输入合并层

玩家键盘写 `K`，AI 写 `AIK`，两者都是同形状的布尔值对象， 在 `inp()` 里按位或合并。这样做的关键不是省代码，而是**让「AI 有没有作弊」变成一个可证伪的命题**： AI 没有自己的移动函数、没有自己的碰撞函数，它能做的只有「按住某个方向」。

```
const K   = { left:false, right:false, up:false, down:false, fire:false };
const AIK = { left:false, right:false, up:false, down:false };

function inp() {
  return {
    left:  K.left  || AIK.left,
    right: K.right || AIK.right,
    up:    K.up    || AIK.up,
    down:  K.down  || AIK.down,
  };
}
```

3.4 二倍速：为什么不写 `update(dt * 2)`

这是本项目里最容易写错、也最值得写清的一处。灵弹每帧位移 9px，符块短边 34px， 球与符盘的判定半径 8.5px。**一步走两倍距离，就会直接穿过符块和挡板** —— 静止物体不会「挡」住它， 因为中间那一帧根本不存在。所以二倍速必须写成「多跑几帧」：

```
function tick(dtFrames) {
  run.acc += dtFrames * run.speed;      // speed = 1 或 2
  let guard = 0;
  while (run.acc >= 1 && guard++ < 32) {   // ⚠️ guard 上限别设太小
    update(1);                            // 物理步长永远是 1
    run.acc -= 1;
  }
  return guard;
}
```

这条代码被自己的判据坑过一次

`guard++ < 8` 时，判据驱动 `tick(10)` 只会推进 8 步 —— 看起来像「二倍速没生效」，实际是判据假设了实现的内部上限。**判据不该假设被测实现的任何上限。** 现在上限放到 32（真实显示帧最多 6 步，仍能防死循环），并在判据里显式比对 1× 与 2× 的推进步数。

3.5 渲染分层

五款的 `draw()` 次序几乎一致：**背景** → **世界实体** → **弹道/特效** → **HUD** → **边框/暗角** → **边缘脉冲**。 两个细节值得单说：一是 HUD 与世界之间必须**裁剪**（否则符块会压到血条上）， 二是受击反馈只用「屏幕边缘的径向渐变」—— 中心 alpha 恒为 0，只有四周泛色，没有任何整屏闪烁或位移。

```
function drawEdge() {
  const v = run.edge.v;
  if (v <= 0.01) return;
  const g = ctx.createRadialGradient(W/2, H/2, H*0.30, W/2, H/2, H*0.74);
  g.addColorStop(0,    'rgba(' + run.edge.rgb + ',0)');            // 中心完全透明
  g.addColorStop(0.62, 'rgba(' + run.edge.rgb + ',' + (0.06*v) + ')');
  g.addColorStop(1,    'rgba(' + run.edge.rgb + ',' + (0.30*v) + ')');
}
```

04 逐款实现拆解

> 下面五节只写每款「真正有技术含量、换一款游戏就不成立」的部分。玩法说明一律略过。

4.1 星际防线 · 初版：把参数从代码里抽出来

这是五款里唯一使用 ES class 与集中配置表的。13 个类（Player / Bullet / Enemy / Particle / Star / Nebula / Meteor / Shockwave / FloatText / PowerUp / EnemyBullet / Boss + Game）分得很干净， 而 `CFG` 把 SPEED / AI / PLAYER / DASH / TOUCH / WEAPON / BOARD / COMBO / POWERUP / BOSS / FX / IMPACT / METEOR / BULLET / ENEMY / DIFFICULTY / PARTICLE 共十七组参数全部外置。

这带来的具体好处：**三把武器不是三段逻辑，而是同一张表的三行。**改手感不用读代码， 改 `CFG.WEAPON.LASER` 就行。难度推进也是显式阶梯而不是散落各处的随机：

```
// 难度按时间阶梯上升，而不是"越玩越难"这种无法验证的说法
level = clamp(1 + Math.floor(elapsed / 16), 1, 12);
```

代价也真实存在：5600 行、240KB，是这个仓库里最难改的一款。它后来基本只做 UI 提亮与去抖屏， 没有再动玩法 —— 因为动一次就要重新理解 13 个类的生命周期。



星际防线 · 初版：800×600，AI 接管中。背景保留深色（太空射击背景一亮就毁观感），只把面板与 HUD 提亮。

4.2 符箓灵弹 • 巧阵：把「下压」做成可计算的墙

v1.3.0 整款重做。旧版是「敌人一边下压一边喷满屏符雨」，玩家的操作只剩左右躲，弹幕一密就变成 「看不清、躲不掉、靠运气」。新版把威胁从「看不清的子弹」换成「空间被一点点吃掉」： 一面 8 列 × 2~5 排的符阵墙自上方压下，玩家用符盘接住灵弹、调倾角把它敲穿。

几何比遍历更重要。球撞块时用最小重叠轴分离，并在处理完一块后立刻 `break` —— 否则一帧内会连着反弹两块，方向乱跳：

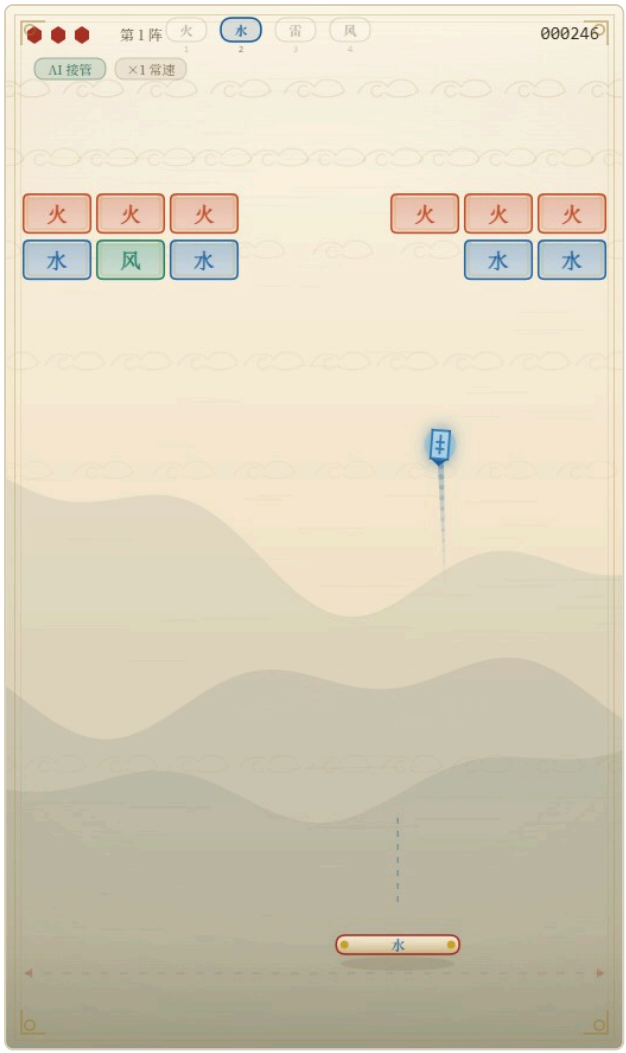
```
// 找最小重叠轴：只沿"穿得最浅"的那一维推出去
const ox = (BALL_R + b.w/2) - Math.abs(bx - cx);
const oy = (BALL_R + b.h/2) - Math.abs(by - cy);
if (ox < oy) { ball.vx *= -1; ball.x += bx < cx ? ox : -ox; }
else         { ball.vy *= -1; ball.y += by < cy ? oy : -oy; }
return;   // ⚠️ 一帧只处理一块，否则连续反弹两块
```

元素反应：插在伤害结算最前面的一张表

```
const RX = {
  'hydro+pyro':    { name:'蒸发', mul: 2.0, kind:'vapor' },
  'pyro+hydro':    { name:'蒸发', mul: 1.6, kind:'vapor' },
  'pyro+electro':  { name:'超载', mul: 1.5, kind:'aoe'   },
  'electro+hydro': { name:'感电', mul: 1.3, kind:'chain' },
};

function reactionOf(atk, def) {
  if (atk === 'anemo') return { name:'扩散', mul: 1.25, kind:'swirl' };
  if (atk === def) return null;              // 同元素不反应
  return RX[atk + '+' + def] || null;
}
```

反应不是「弹个字」，而是**走真伤害**：蒸发改变倍率；超载在 104px 半径内炸到邻块（14 点）； 感电对同排最近 3 块各 10 点并画电弧；扩散把邻近 3 块的元素「染色」成攻击元素。 这一点用判据卡死了：同样的三块符牌、同样的初始血量，同元素 90 伤、水打火 80 伤、火打水 84 伤。



符箓灵弹 · 巧阵：宣纸暖白底 + 描金装裱框，符块是玉牌质感，每种元素一色。左上角是常驻可点的「AI 接管 / ×2 二倍速」徽标。

AI：预测落点 + 反解倾角

AI 做三件事，全部只写方向键：① `predictLanding()` 逐帧模拟侧墙与顶墙反弹，求出球落到 符盘高度时的 x；② 球接近符盘时用打分函数选目标块 （`得分 = 反应倍率×46 − 块的 y×0.055 − 水平距离×0.045 − 血量×0.55`），再用 `atan2` 反解出符盘需要的倾角；③ 每 20 帧换一次元素。

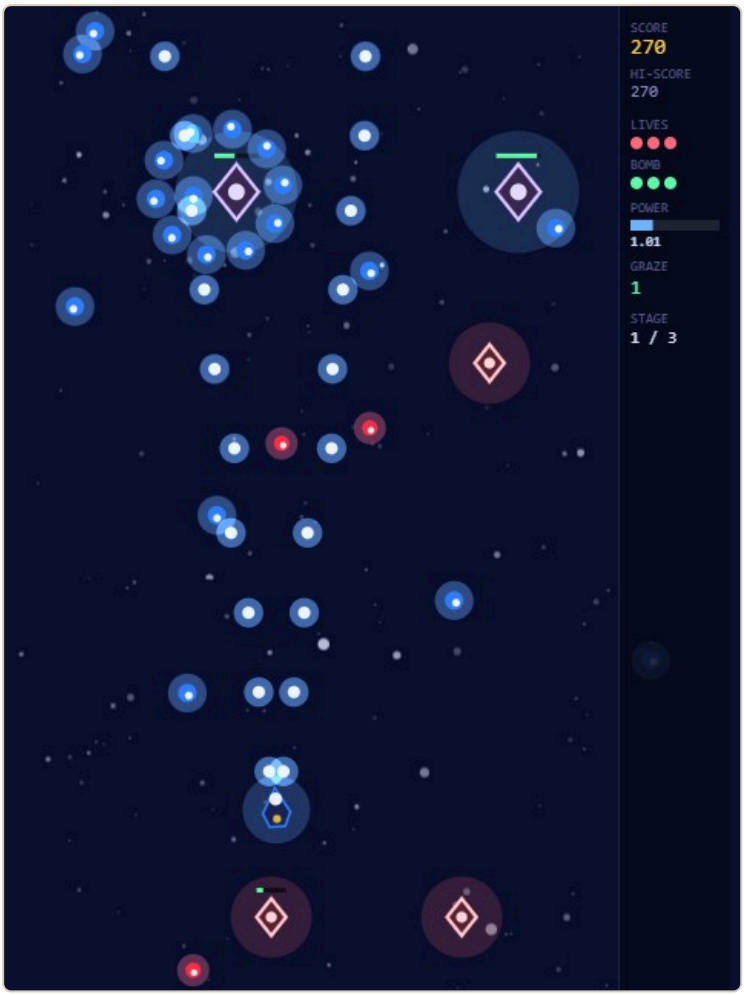
4.3 星屑符咒 • 弹幕手记：不采样，直接解方程

弹幕 AI 的常见写法是「往 16 个方向各射一条射线，挑最安全的」。这款没这么做，因为采样会有盲区， 而且结果对采样角度的选择很敏感。它用的是**相对运动下的闭式最近接近点**：

```
// 弹与人的相对位置 r、相对速度 v（都在"以玩家为原点"的坐标系里）
// 距离平方 |r + t·v|² 对 t 求导 = 0  →  t* = −(r·v) / |v|²
const tStar = −(rx*vx + ry*vy) / (vx*vx + vy*vy);
const tt = Math.max(0, Math.min(tStar, HORIZON));   // 只关心未来一小段时间
const d  = Math.hypot(rx + vx*tt, ry + vy*tt);      // 那一刻的最近距离
```

有了这个量，整屏的威胁就不再是「一堆子弹的坐标」，而是每颗弹一个标量。 决策层用 17 个候选方向，**先按安全余量筛、再按目标方向排序**，而不是加权求和 —— 加权和会让「为了瞄准而站着不动」反过来吃掉安全性，这是 v1.1.0 实测踩到的。 容差随局势在 12 / 6 / 2 三档自适应：宽裕时敢走位，被逼到墙角时只求活。

v1.3.0 的弹幕重做不是「弹数砍一半」，而是保证**任何一帧都留有一条可走位通道**： 每张符卡留预告期、弹速分层（慢弹封路 / 快弹逼迫）、禁止多组互相矛盾的封路弹同时出现。 量化前后最差安全格 9.4% → 53.8%，AI 9/9 通关，存活中位 178 秒。



星屑符咒 · 弹幕手记：480×640 竖版。重做后弹幕分层，慢弹负责封路、快弹负责逼迫，中间始终留通道。

4.4 深渊回响：房间不是迷宫，是一条链

这类 roguelite 很容易上来就写房间图生成（BSP / 随机游走 / 图搜索）。这一版没有： **房间就是一个线性数组，每间都是同样的 16×10 格**，「进下一间」是重建整张 map、 再把玩家瞬移到中心。牺牲了地图的拓扑感，换来的是「任何一层都能被完全复现」—— 对判据来说这是决定性的。

移动用的是同帧双轴位移 + 逐轴回退：先按 `(dx, dy)` 一起走，撞墙就分别把 x、y 退回来。 比「先 X 后 Y 分两次移动」少一次穿透窗口，贴着墙角斜走时不会卡住。

v1.3.0 的「一关之后还有下一层」不是加一个循环就完事，而是必须让层级有意义： `run.depth++` 之后**房间数与怪物血量同时上涨**，否则层数只是个数字。 判据断言「清完最后一间房 → depth+1 且游戏未结束」以及「连续下潜 3 层不崩」。



深渊回响：640×448 俯视角。地牢保留阴郁基调，只把面板与文字提亮，避免和符箓灵弹的暖白撞成一套。

4.5 星轨防线 v2：缝合三套系统而不打架

这一版是在初版基础上「缝」进关卡链、升级、炸弹、多武器。缝合最容易出的问题是 **升级直接改玩家字段，导致无法回滚、无法对比。**它的做法是引入一张派生表：

```
// 升级不改玩家，只改"推导表"；要重置整局，重算一遍就行
function weaponStats() {
  const u = run.upg;                        // 已获得的升级计数
  return {
    dmg:   6 * (1 + 0.18*u.dmg) * powerMul(),
    cd:    Math.max(3, 9 - u.rate),
    shots: 1 + (u.multishot ? 1 : 0) + (u.spread ? 2 : 0),
    spread: u.spread ? 0.34 : 0.06,
  };
}
```

另一个细节是**护盾朝向**：`isFromBack()` 判「弹道方向与盾法线夹角」， 背后来弹 ×2.5 伤害。它让「转身」这个动作第一次有了战术含义 —— 不是加数值，是加决策。



星轨防线 v2：720×480。升级三选一、炸弹、多武器并存，数值全部由 `WS` 派生表推导。

05 借鉴来源

> 五款游戏各自站着一个明确的参照系。下面逐条写清**借了什么、改了什么、没碰什么** —— 尤其是最后一条：本项目没有使用任何第三方模型、贴图、字体或音效素材，画面全部是 Canvas 程序化绘制。

5.1 一表看清

参照作品	用在哪款	借的机制	本作的改造
原神 Genshin Impact	符箓灵弹	火 / 水 / 雷 / 风四元素；蒸发、超载、感电、扩散的「反应」概念；璃月系配色（暖纸、描金、朱砂、青玉）	反应做成**伤害倍率表**并挂上副作用（范围炸、同排连锁、元素染色）；配色只取语言，纹样全部程序化重绘
东方 Project Touhou	星屑符咒	纵版弹幕的密度语汇、符卡（spell card）与预告期、道中 + BOSS 的关卡结构	把「难点是弹幕美学」改成「难点是可走位性」：每张符卡都必须留可通行解，用安全格占比量化
元气骑士 Soul Knight	深渊回响	房间清怪、宝箱、升级三选一、一层层往下潜的循环	去掉随机地图拓扑，改线性房间链以换取可复现；下潜时同时抬房间数与怪物血量
打砖块 Breakout / Arkanoid	符箓灵弹	挡板接球、按落点改变反弹角的「角度控制」、砖墙成排下压	把「砖」换成带元素的符块、把「挡板」换成可旋转的符盘、把「下一关」换成「下一阵」
雷电 Raiden 系列	初版 / 星轨 v2	纵版射击的三武器、道具与火力升级、BOSS 三阶段血线（66% / 33%）	初版把三武器做成同一张配置表的三行；v2 加护盾朝向与背击判定
吸血鬼幸存者 Vampire Survivors	星轨 v2	「升级 → 三选一 → 立刻变强」的节奏循环	升级只改派生表 `WS`，随时可整体重算，方便对比不同 build

5.2 原神的元素反应：借机制，不借数值

元素反应是本项目里「借鉴」二字最集中的地方，也因此最需要说清边界。原作的反应是 **固定伤害 + 抗性/精通修正**，而符箓灵弹的符块只有血量没有抗性，直接照搬会让数值失去参照。 所以这里只保留**反应的方向性与相对强弱**，把量级重排成一套能和自己血量配合的倍率：

反应	触发（攻击 × 目标）	本作倍率	本作副作用
蒸发	水 → 火	`×2.0`	单体高伤，无附加效果
蒸发	火 → 水	`×1.6`	同上，但弱一档，形成「先上水」的策略倾向
超载	火 × 雷	`×1.5`	104px 范围内邻块各 14 点，适合拆密集阵
感电	水 × 雷	`×1.3`	同排最近 3 块各 10 点，并画电弧
扩散	风 × 任意	`×1.25`	邻近 3 块元素被「染色」成攻击元素，铺路用

必须说清的取舍

上面这组倍率是**为本作节奏重排的**，不是原作的官方数值；原作里超载 / 感电 / 扩散是固定伤害， 这里改成倍率才能和符块血量配合。参考的只是公开资料里「谁触发谁、谁强谁弱」的结构。 另外，美术没有取用任何官方资源 ——「贴合美工」是通过**配色语言和装裱式排版**做到的： 宣纸暖白底、双细金线 + 四角云勾、玉牌质感的符块、带符头的纸符灵弹、朱砂虚线的阵线。

5.3 星屑符咒与东方：把「弹幕美学」拆成可验证的约束

东方式的弹幕游戏，难点往往在于「好看」与「可躲」是同一件事：弹幕要密到有压迫感， 又必须留出一条窄缝。照抄画面没用，必须把这条窄缝变成可计算的量。做法是：

- **预告期**：任何封路弹在真正构成威胁前，必须有固定帧数的可视预警（光柱 warn 60~66 帧）。
- **弹速分层**：慢弹负责「封路」（占位），快弹负责「逼迫」（驱赶）。两层同时出现才算压力， 单层只算装饰。
- **禁止矛盾封路**：两组弹幕不允许在同一帧把左右两侧同时锁死 —— 这一条直接写进了生成器。

BOSS 登场即清场也是从这里借的惯例：它给玩家一个「正戏开始」的信号，同时避免道中残留弹 和 BOSS 弹叠加成无解局面。

5.4 深渊回响与元气骑士：删掉一半系统

元气骑士的核心循环（清房 → 开门 → 宝箱 → 升级 → 下一层）被完整保留，但地图生成被大幅简化。 这不是偷懒，是**明确的取舍**：随机地图让每一局都不同，可它也让「这一层为什么打不过」变得无法归因。 线性房间链 + 固定尺寸的结果是：任何一层的失败都能被复现、被度量、被修。

5.5 符箓灵弹与打砖块：一个被低估的机制

打砖块真正好玩的地方从来不是「打掉砖」，而是**用落点控制反弹角**。这个机制在符箓灵弹里被放大成 核心操作：符盘可以旋转（`↑↓`，倾角上限 ±0.95 rad），球撞上符盘时的出射角由倾角决定。 于是「接住」只是及格，「接到哪里」才是技术 —— 而 AI 反解倾角的那一步，也正好是这个机制的镜像。

素材与版权

五款游戏的图形全部由 Canvas 2D API 程序化绘制（矩形、路径、渐变、噪声）， 没有位图素材、没有外部字体文件、没有音频文件（音效全部为 WebAudio 合成或干脆留空）。 借鉴的是**玩法机制与视觉语言**，不是资源。仓库 README 里长期保留「美术与角色全部原创」的声明。

06 验证方法论

> 这个项目最不像「个人练习作」的地方，是它有一套能自己给出结论的验证体系： 183 项断言、21 秒跑完、五款全绿。但更值得写下来的是**判据自己错的那几次。**

6.1 判据分三层

层	问的问题	例子
硬断言	机制有没有被触发?	「敲光最后一块 → `wave` 从 1 变 2」「掉到阵线以下 → `hp` 3→2 且球回到符盘」
数值对照	效果是不是它声称的那个量?	三块同血量符牌：同元素 90 伤 / 水打火 80 伤 / 火打水 84 伤
统计量	多跑几轮还成立吗?	AI 连打 3 轮、每轮 90 秒，每轮至少破 2 阵；星屑符咒 5 轮里 4 轮通关

判据全部跑在真实浏览器里（headless Edge + CDP），通过各游戏导出的 `window.__TP` / `__DM` / `__SK` / `__SL` / `__SpaceLine` 接口驱动。 没有 jsdom 之类的模拟环境 —— 因为要验的东西里有一半是「画出来对不对」。

6.2 「AI 不作弊」怎么被证伪

「AI 能通关」这句话在没有约束时毫无意义：随便给个瞬移也能通关。所以每款游戏都逐项比对 开关 AI 前后的物理常量，并实测单帧位移：

检查项	玩家	AI	判定
判定半径	8.5	8.5	相同
擦弹半径（星屑符咒）	12	12	相同
无敌帧	72 帧	72 帧	相同
移动速度	7.8 px/帧	7.8 px/帧	相同
实测单帧位移	≤ 7.8	≤ 7.8	未超过玩家速度

这条约束也直接决定了 AI 的实现方式：它只能写 `{left,right,up,down}` 四个布尔值， 没有任何「设置坐标」「设置速度」的通道。

6.3 截图能抓到判据抓不到的四件事

判据全绿之后，我仍然把每一款的成品截图看了一遍 —— 抓到的全是「数据正确、观感错误」：

现象	原因	修法
每阵开头有 5 秒空场	墙生成在 `y = -122`（屏幕外），要飘 5 秒才进画面。判据只验「10 秒下移 110px」，完全合法	改为从可见区顶部 `y = 82` 淡入（22 帧）
符块压住血条与元素条	两者都在画布同层绘制，没有裁剪	`drawWall()` 内 `ctx.rect(0,76,W,H-76)` + clip
提示条近乎常亮还压住符墙	AI 每 20 帧换元素，而 `setEl()` 无条件弹提示	给 `setEl(i, silent)` 加静默参数，AI 走静默通道
想点开 AI 却点不到	徽标原本 `if (AI.on) drawBadge(...)`，关闭时不画	常驻绘制 + 画布点击命中（按 CSS 缩放比换算回逻辑坐标）
灵弹看着像个医疗十字	三笔画（中轴 + 两横）+ 方框，视觉上就是 ✚	改成纸符比例（宽 0.86R、高 1.40R）+ 顶部符头三角，旋转量减半

6.4 判据自己错了三次

三次都是「判据错、游戏对」

- **在 `start()` 之前取了 `run` 引用。** `start()` 内部会 `newRun()` 新建对象，先取引用拿到的是上一局的尸体 —— 16 条断言同时红， 看起来像游戏彻底崩了。此后所有判据统一写成 `D.start(); D.freeze(); const R = D.run;`，并把这条约定写在文件头。
- **把被测对象摆在屏幕外。**验「BOSS 符矢同时在场 ≤ 4」时，符核被放在 `wallY = -260`，符矢生成即出界被过滤，峰值永远读到 0。改成 `wallY = 120` 后峰值 3。
- **把 1× 与 2× 的期望值写反了。**累加器本身没错，是断言里 `d1`（1×）写成了 20、 `d2`（2×）写成了 10。

还有一次是「判据没错、实现也没错，只是语义不同」： `guard++ < 8` 让 `tick(10)` 只推进 8 步，看着像二倍速失效。结论写成一句规则： **判据不许假设被测实现的任何内部上限。**

07 缺陷档案

> 下面这张表里的每一条，都是「判据全绿」之后仍然存在过的真 bug。它们有个共同特征： **逻辑在跑、玩家感知不到**，或者**感知到了但归因错了地方**。

7.1 已修

现象	根因	修法
球永远落不到底线（玩家上一轮就抱怨过手感）	「角度回正」在球位于符盘下方时，把纵向速度也拽向斜上方；而「落底自动回收」正是核心机制。 对应判据一直红，被当成「测试写太严」放过去了	重写回正逻辑只作用于水平分量；该判据改为硬断言
敲光一墙后卡住不换阵	`destroyBlock()` 只置 `alive = false`，数组里仍留着，`blocks.length === 0` 永不成立	`update()` 内过滤 `run.blocks = run.blocks.filter(b => b.alive)`
同一波可连扣三下血	`hurtCd` 以秒记（1.15）而 `dt` 以帧记，漏乘 `0.0166`，冷却只维持 1 帧	`run.hurtCd -= dt * 0.0166`
越线扣命取错排	遍历 `blocks` 取第一个越线块（最上一排）就 `break`	改为取 `row` 最大者（最下面那排）
页面一加载墙就在背后压下	加载后直接 `requestAnimationFrame(frame)`，没等玩家点开始	加载只 `draw()` 一张静态底图，点「开始」再启循环
敌人按 `time-to-hit` 推进时 AI 宁可漏怪	把「站到敌机下方开火」判成高危（−33 分 vs 击杀收益 1.34），结果 **87% 的掉命来自漏怪**	敌机改用 0.5 秒三次衰减的威胁模型 —— 威胁可消除时，躲避模型会得出保守到自毁的策略
深色主题的强调色搬到浅色底上等于看不见	亮色强调色在白底上对比度不足，整套视觉层级塌陷	整组重挑，按 WCAG 复核 ≥ 4.5:1

7.2 尚未修复：深渊回响的三处实现缺口

这三条是在做本文的实现拆解时，用「定义 → 调用」交叉核对查出来的，**判据没有覆盖到**：

#	缺口	后果
1	`drawEBullets()` 定义在 1481 行，但 `draw()` 的调用表里**没有它**	法师和 BOSS 的子弹真实存在、真实扣血，屏幕上完全看不见 —— 玩家只会莫名掉血
2	`pl.rx / pl.ry` 赋值后从不清零，而撞墙回退量用的是手操位移 `ax*spd*dt`	翻滚时基本不撞墙；探针方向永远停在最后一次翻滚的方向
3	`b.delay` 全文件没有任何递减语句	法师「三连点」实际只打出 1 发，另外两发永久隐身飞出屏幕

为什么先不修

三条修完**都会让游戏变难**：凭空伤害变成可见威胁、法师真的会三连。这不是纯 bugfix， 而是平衡改动，需要同步复核 44 项判据并补两条针对性断言（「敌弹必须被绘制」「三连点必须打出三发」）。 修之前先记在这里，避免下次又靠人眼才发现。

08 交付与运行

8.1 桌面归档目录

```
Canvas游戏项目-2026-10/
├─ README.txt                怎么玩、怎么跑判据
├─ games/                    完整工程（可直接双击各 index.html）
│   ├─ index.html            档案馆首页（版本记录）
│   ├─ manifest.json         五款元数据 + 五个版本的 notes
│   ├─ space-line-v1/        初版    + verify/smoke.js（14 项）
│   ├─ talisman-pinball/     符箓灵弹 + verify/smoke.js（29 项）
│   ├─ touhou-danmaku/       星屑符咒 + verify/smoke.js（40 项）
│   ├─ soul-knight/          深渊回响 + verify/smoke.js（44 项）
│   ├─ starlane-v2/          星轨 v2  + verify/smoke.js（56 项）
│   ├─ _run_all.js           一条命令跑完 183 项
│   └─ versions/             五个版本的历史快照
├─ 文档/
│   ├─ 开发过程与借鉴分析.pdf      ← 本文
│   └─ 实现解析.html             五款源码级解析（带行号）
└─ 演示/
    ├─ 01_星际防线初版.mp4  … 05_星轨防线v2.mp4
    └─ 演示合集.html          一页看完五段
```

8.2 怎么跑

想做的事	怎么做
直接玩	双击 `games/<游戏>/index.html`；或用任意静态服务器起一个更稳
跑全部判据	`cd games && node _run_all.js`（约 21 秒，输出 183 项结果）
跑单款判据	`node games/<游戏>/verify/smoke.js`
看历史版本	`games/versions/<日期>/<游戏>/index.html`

判据脚本依赖一个真实的 Chromium（Edge/Chrome）与 CDP，不依赖 npm 包；它会把页面加载到 `file://` 或本地 http，再用各游戏导出的测试接口驱动。

8.3 演示视频怎么录的

这一段本身也是个技术坑，值得记下来：在 headless 下，`canvas.captureStream(30)` 只在页面真正被合成时推帧，于是录出来是**一张静帧被拉长成 3.37 秒**。 最终做法是：有头窗口（保证真实 60fps，游戏速度才对）+ `captureStream(0)` 关闭自动推帧 + 每 33ms 手动 `track.requestFrame()`，用 MediaRecorder 直接封装 H.264 MP4。

```
const stream = cv.captureStream(0);        // 0 = 别自动推帧
const track  = stream.getVideoTracks()[0];
const rec    = new MediaRecorder(stream, { mimeType: 'video/mp4;codecs=avc1.42E01E' });
rec.start(200);
setInterval(() => track.requestFrame(), 33);   // 由我们决定"什么时候是一帧"
```

09 附录：文件地图

路径	规模	作用
games/space-line-v1/index.html	5,619 行	初版：13 个类 + CFG 十七组配置
games/talisman-pinball/index.html	1,619 行	符箓灵弹：符阵墙 + 四元素反应
games/touhou-danmaku/index.html	2,334 行	星屑符咒：闭式威胁评估 + 17 方向候选
games/soul-knight/index.html	1,990 行	深渊回响：线性房间链 + 无限下潜
games/starlane-v2/index.html	2,366 行	星轨 v2：派生表 + 护盾朝向
games/<游戏>/verify/smoke.js	合计 183 项	无头浏览器回归判据
games/_run_all.js	—	全仓一条命令回归（21 秒）
games/_build_site.js	—	由 manifest 生成档案馆首页
games/_snapshot.js	—	把当前五款 + 日志冻结成一个版本快照
games/_tools/ · _reports/	—	平衡性探针与它们跑出来的报告
games/manifest.json	5 版本	版本、日期、摘要与逐条 notes
games/versions/2026-10-19/	5 款 + 日志	v1.3.0 完整快照
games/docs/实现解析.html	11 章	逐款源码解析（更细的行号与片段）

9.1 一句话总结

这五款游戏真正的「作品」不是画面，而是那套让画面可以被质疑的机制： **把感受翻译成指标，把指标写成判据，把判据跑成习惯。** 15 秒的平衡探针、183 项断言、五次版本快照，本质上都在做同一件事 —— 让「我觉得这样更好玩」变成「我这里差 44.4 个百分点」。

文档生成于 2026-10-20 · 对应代码版本 v1.3.0（提交 a7d9c3e）
全部截图与演示均来自本机实机运行，非渲染图或示意图。